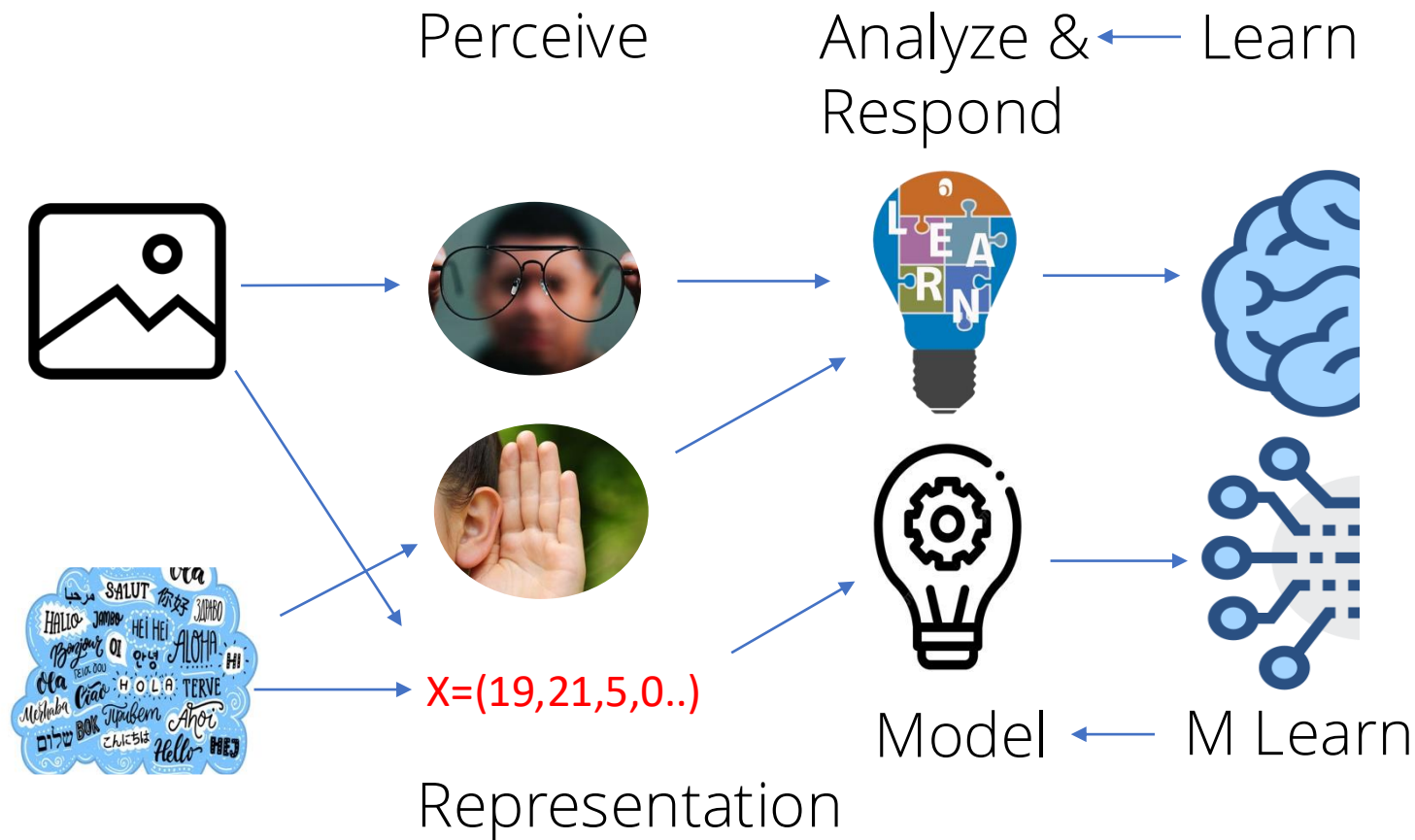


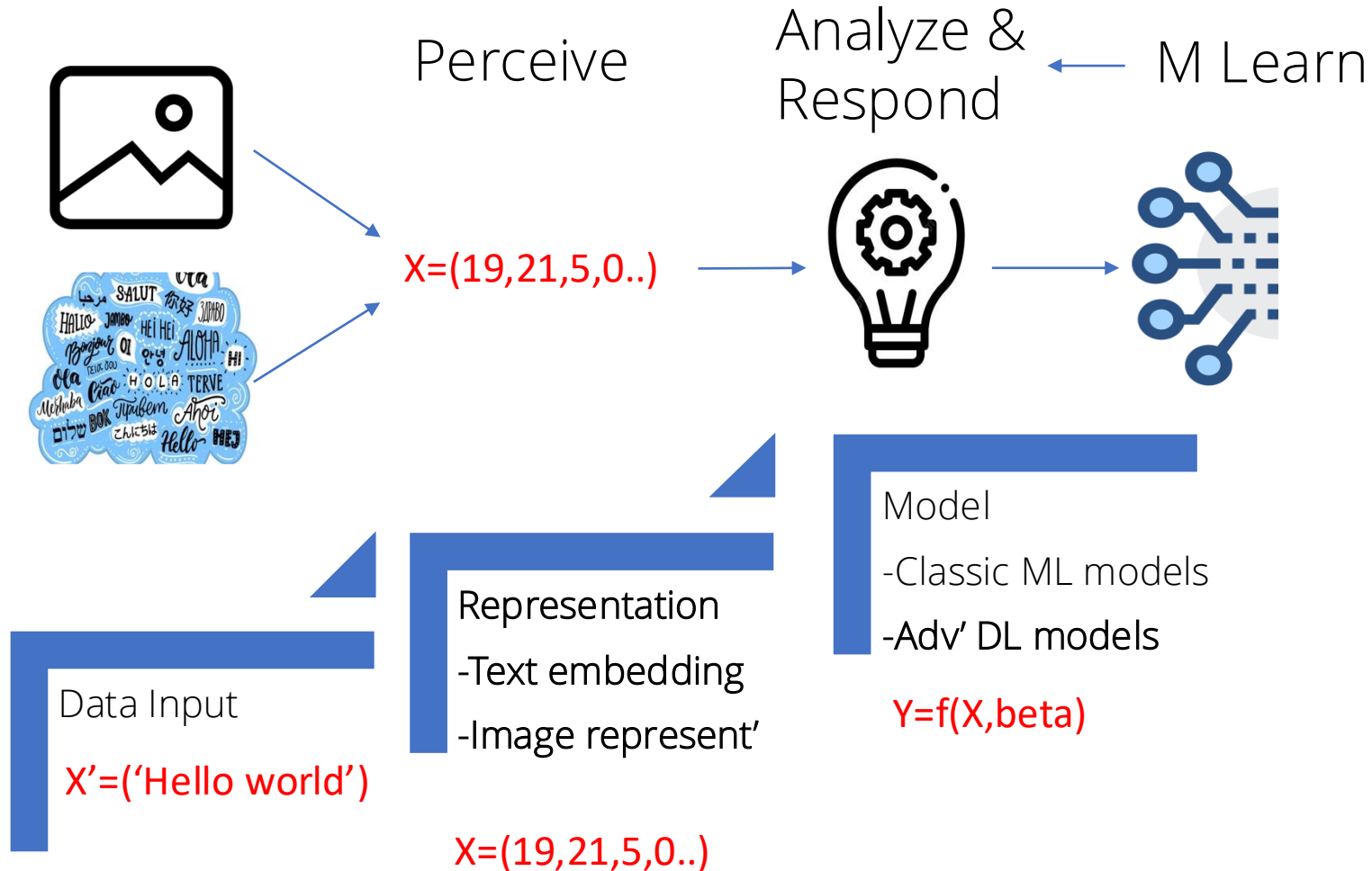
S2: Text Representation

Unstructured data analytics process

- Artificial Intelligence on unstructured data



Unstructured data analytics process



Objective of representation

- Transform into a format that computers can analyze
 - Numerical, a lot of options
- Preserve as much information as possible
 - Differs across text and image
 - Differs across different “needs”

Agenda

- Text representation
 - Tokenization
 - Document/sentence embedding
 - Bag of words
 - Stemming
 - Lemmatization
 - TF-IDF
 - Phrase level information
 - N-grams
 - Part-of-speech level information
 - POS-tagging
 - Word embedding
 - Index and one-hot encoding
 - Word embedding
 - Word2Vec/GloVe

What is text?

- Example: IBM Corp. and Alvarion Inc. have established an alliance to offer wireless systems to municipalities and their public safety agencies, Alvarion announced.
- Text document as a sequence of
 - Characters
 - Words
 - Phrases
 - Part-of-speech
- It seems natural to think of a text document as a sequence of **words**
 - A fine-grained unit
 - Naturally separated by spaces or punctuation
 - Meaningful (character is not)

Tokenization

- Language text
 - d1 = "David loves dogs"
 - d2 = "Dogs are ok with David"
 - d3 = "Cats love rain"
- Tokenization: split sentences into pieces, called *tokens*
 - Note: token can be more general, it can be a character, a word, a sentence, etc.
- When a token is a word, it results in a set of words
 - d1 = ["david", "loves", "dog"]
 - d2 = ["dogs", "are", "ok", "with", "david"]
 - d3 = ["cats", "love", "rain"]

Python: Tokenization

- There are many tokenizers based on different rules
- <https://www.nltk.org/py-modindex.html>

```
import nltk

d1 = "Jinyang is a good guy, he is not bad"
d2 = "feet wolves cooked boys girls ,!<@!"
d3 = "Jinyang is not a good guy, he is bad"

c1 = [d1, d2, d3]

#TOKENIZE
token_d1 = nltk.word_tokenize(d1)
print(token_d1)

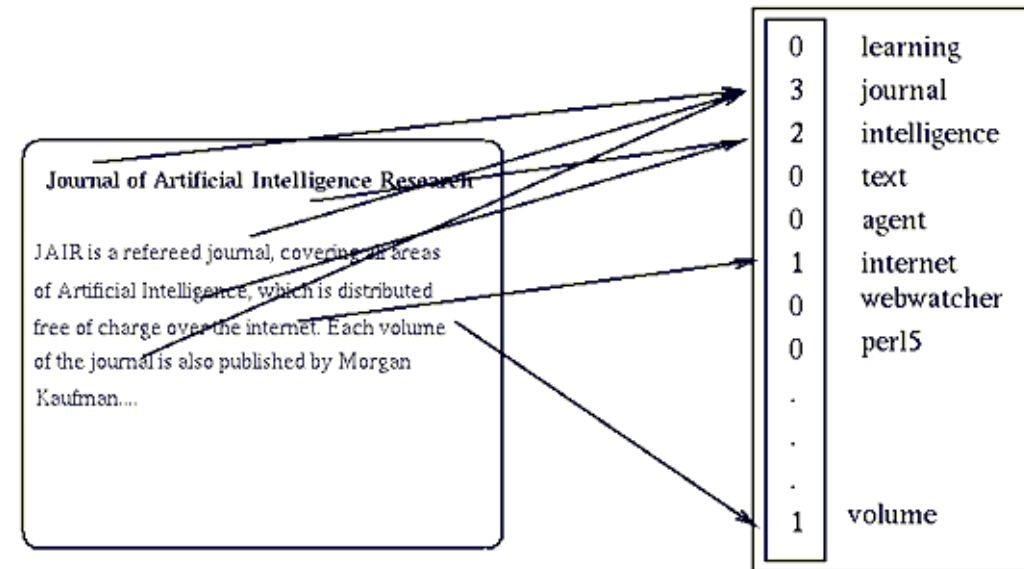
tokenizer2 = nltk.tokenize.WhitespaceTokenizer()
Token_d12 = tokenizer2.tokenize(d1)
print(token_d12)
```

Discussion

- Does tokenization yield a numerical representation?
- It is still text!! Can't use as numeric (e.g., vectors) to compute, manipulate and analyze
 - `s1 = ["david", "loves", "dog"]`
 - `s2 = ["dogs", "are", "ok", "with", "david"]`
 - `s3 = ["cats", "love", "rain"]`
- We need to transform text into a numerical representation, so we can apply data mining or ML/AI methods to them

Document/sentence embedding

- A simple representation: Bag of Words (BOW)
- A naïve idea is to count the frequency of words
- Define an ordered vocabulary
- One **document** is represented as one **vector**
- Inter-word independency
- No order/sequence
- Enough for extracting topics or basic sentiment analysis
- Why 0s?
- The vocabulary is defined to include an unduplicated list of words in the whole **collection** of the documents, why?



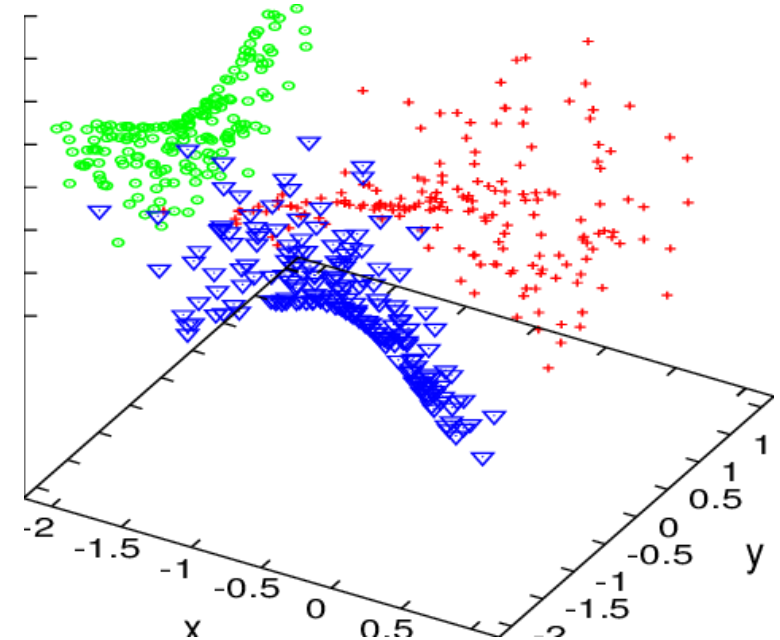
Term-document matrix

- A classic form of BOW output
 - Term: typically, a single word, or an **n-gram** phrase like “data mining”, vocabulary
 - Document: one sample, the unit of sample text object
 - Can be large - terms are often 50k or larger, documents can be in the billions (www).
 - Can be binary, or use **counts**
 - Example: 5 doc, 6 terms $D_1 = (d_{i1}, d_{i2}, \dots, d_{it})$

	Database	SQL	Index	Regression	Likelihood	linear
D1	24	21	9	0	0	3
D2	32	10	5	0	3	0
D3	12	16	5	0	0	0
D4	6	7	2	0	0	0
D5	43	31	20	0	3	0

Euclidean visualization

- Each term is represented as a separate **variable** having numeric weight (importance) as frequency/dummy
- When $|\text{terms}|=3$, each document is a dot in a 3-d Euclidean space
- Two different documents are two different dots
- The more different, the farther the **distance**



Python: BOW term-document matrix

- Use a term-document matrix for a simple bag-of-words representation

```
##BOW Frequency
from sklearn.feature_extraction.text import
CountVectorizer
vectorizer1 = CountVectorizer()
vectorizer1.fit(c1)

print(vectorizer1.vocabulary_)

v1 = vectorizer1.transform(c1)
print(v1.toarray())
```

High-dimensionality issue

- Source 1: **different forms** of a word but the **same meaning**
- Different spelling and similar meaning
 - E.g., learn, learns, learned, learning
 - E.g., dogs, dog
- Solution 1: **normalization**
 - Use the same token for different forms of the same word
 - Approach 1: stemming
 - Approach 2: lemmatization
 - ...

Stemming

- A process of transforming a word into its stem, base or root form (normalized form) by removing its affix
- Example: Porter's stemmer `nltk.stem.PorterStemmer`
<http://www.tartarus.org/~martin/PorterStemmer/>

Step 1a

sses	→ ss	caresses	→ caress
ies	→ i	ponies	→ poni
ss	→ ss	caress	→ caress
s	→ ∅	cats	→ cat

Step 1b

(*v*)ing	→ ∅	walking	→ walk
		sing	→ sing
(*v*)ed	→ ∅	plastered	→ plaster

Step 2 (for long stems)

ational	→ ate	relational	→ relate
izer	→ ize	digitizer	→ digitize
ator	→ ate	operator	→ operate

Step 3 (for longer stems)

al	→ ∅	revival	→ reviv
able	→ ∅	adjustable	→ adjust
ate	→ ∅	activate	→ activ
...			

Stemming and dimension reduction

- Dimension Reduction

	Tokens = N	Types = V
Switchboard phone conversations	2.4 million	20 thousand
Shakespeare	884,000	31 thousand
Google N-grams	1 trillion	13 million

- Limitation: irregular forms and produce non-words
 - dummies ~ dummi
 - feet ~ feet

Lemmatization

- Use a pre-defined database (dictionary) to lookup lemmas
- Example: WordNet lemmatizer `nltk.stem.WordNetLemmatizer`
 - dummies ~ dummy
 - feet ~ foot
- Limitation
 - not for all words (cooked ~ cooked), and it is slower
- Which is better:
 - try both and find the better
 - or combine them for your task

Other normalization

- Capital letter: lowercase
 - Special case: Us, us, US
- Punctuation (.,!@#\$%^&*~|}{_+:"'?><` ;'[]\): filter out
 - Special case?
- Acronyms
 - US, U.S, USA, usa, U.S.A ~ U.S.A
- Case by case, write case-specific rules for treatment
- Don't be a Perfectionist

Python: Normalization

- There are many stemmers and lemmatizers based on different rules
- <https://www.nltk.org/py-modindex.html>

```
#STEMMER
token_d2 = nltk.word_tokenize(d2.lower())

stemmer = nltk.stem.PorterStemmer()
stemmered_token_d2 = [stemmer.stem(token) for token in token_d2 if
token.isalpha()]

print(token_d2)
print(stemmered_token_d2)

lemmatizer = nltk.stem.WordNetLemmatizer()
lemmatized_token_d2 = [lemmatizer.lemmatize(token) for token in
token_d2 if token.isalpha()]
print(lemmatized_token_d2)
```

High dimensionality issue cont'

- Source 2: still too many terms
- Solution 2: **drop** less important words
- What kinds of words are less important?
- What kinds of words doesn't help with differentiate a document, considering the frequency?
- Type 1: **high-frequency words**
 - Articles, prepositions, etc. (Examples: A, ABOUT, ABOVE, ACROSS, AFTER, AGAIN, AGAINST, ALL, ALMOST, ALONE, ALONG, ALREADY, IS, ARE...)
 - They are called **stop-words**:
 - Non-linguistic view do **not** carry information: not help to discriminate texts
 - Mainly functional role
- Type 2: **low-frequency terms**
 - Typos
 - Very local information, **over-fitting**

Python: Dimension control

- Remove stop words

```
from nltk.corpus import stopwords

stop_words_removed = [token for token in token_d1 if not
token in stopwords.words('english') if token.isalpha()]

print(token_d1)
print(stop_words_removed)
```

- Remove low-freq terms

```
from sklearn.feature_extraction.text import
CountVectorizer
vectorizer1 = CountVectorizer(min_df=2)
vectorizer1.fit(c1)
print(vectorizer1.vocabulary_)
v1 = vectorizer1.transform(c1)

print(v1.toarray())
```

